

Documento Técnico

Microservicio Inteligente para Gestión de Solicitudes BPO

Paula Rodriguez Monroy

8 de mayo de 2026

Índice

1. Descripción General de la Solución	3
2. Arquitectura Propuesta	3
2.1. API REST (FastAPI)	3
2.2. Orchestrator Service	3
2.3. AI Classification Service	4
2.4. AI Priority Service	4
2.5. AI Justification Service	4
2.6. External Platform Integration	5
2.7. Configuration Layer	5
3. Componentes Desacoplados y Acoplados	5
3.1. Componentes desacoplados	5
3.2. Componentes parcialmente acoplados	6
4. Escalabilidad y Soporte para Nuevos Clientes	6
4.1. Incorporación de nuevas compañías	6
4.2. Nuevas categorías	6
4.3. Reglas de prioridad variables	6
4.4. Integración con plataformas externas	7
4.5. Escalabilidad de volumen	7
5. Estrategia de Inteligencia Artificial	7
5.1. Componentes IA	7
5.2. Justificación de la estrategia	8
6. Variables de Entorno	8
7. Ejecución Local	8

8. Propuesta de Despliegue Cloud 8

8.1. Propuesta cloud 9

9. Conclusión 9

1. Descripción General de la Solución

La solución propuesta consiste en un microservicio desarrollado en Python utilizando FastAPI, diseñado para procesar solicitudes de diferentes compañías clientes mediante el uso de inteligencia artificial y reglas configurables.

El sistema recibe solicitudes en formato JSON, analiza su contenido utilizando un modelo LLM (Large Language Model), clasifica automáticamente la solicitud, determina una prioridad, genera una justificación automática y finalmente delega el caso al área correspondiente según la configuración de cada empresa.

La solución fue diseñada bajo principios de modularidad, desacoplamiento y configuración externa, permitiendo incorporar nuevas empresas, categorías y reglas de negocio sin modificar la lógica principal del sistema.

2. Arquitectura Propuesta

La arquitectura sigue un enfoque basado en microservicios ligeros y separación de responsabilidades.

2.1. API REST (FastAPI)

Responsable de exponer los endpoints HTTP para recibir solicitudes externas.

Responsabilidades:

- Recibir solicitudes
- Validar datos de entrada
- Orquestar el flujo de procesamiento
- Retornar respuestas estructuradas

Archivo principal:

- main.py

2.2. Orchestrator Service

Componente central encargado de coordinar el flujo de negocio.

Responsabilidades:

- Invocar los servicios IA
- Coordinar clasificación y prioridad
- Gestionar delegación
- Integrar servicios externos

Archivo:

- orchestrator.py

2.3. AI Classification Service

Servicio encargado de clasificar automáticamente las solicitudes utilizando un modelo LLM.

Responsabilidades:

- Analizar texto no estructurado
- Clasificar solicitudes
- Utilizar categorías dinámicas según la compañía

Archivo:

- classifier_ai.py

2.4. AI Priority Service

Servicio encargado de determinar la prioridad de la solicitud.

Responsabilidades:

- Detectar urgencias
- Evaluar criticidad
- Aplicar lógica híbrida IA + reglas

Archivo:

- priority_ai.py

2.5. AI Justification Service

Servicio encargado de generar explicaciones automáticas del resultado.

Responsabilidades:

- Explicar la clasificación
- Explicar la prioridad
- Mejorar trazabilidad y auditoría

Archivo:

- justification_ai.py

2.6. External Platform Integration

Componente desacoplado para integración con plataformas externas.

Responsabilidades:

- Simular creación de casos externos
- Facilitar integración futura con CRM, ERP o herramientas BPO

Archivo:

- external_platform.py

2.7. Configuration Layer

La solución utiliza archivos JSON externos para parametrización.

Archivos:

- categories.json
- delegaciones.json

Responsabilidades:

- Configuración por cliente
- Categorías dinámicas
- Delegación configurable
- Evitar hardcoding

3. Componentes Desacoplados y Acoplados

3.1. Componentes desacoplados

Componente	Motivo
Clasificación IA	Puede cambiar de proveedor LLM sin afectar el sistema
Configuración JSON	Nuevas empresas sin modificar lógica
Integraciones externas	Adaptadores independientes
Prioridad	Puede evolucionar a reglas más complejas
Delegación	Configurable por cliente

La utilización de configuración externa reduce el impacto de cambios funcionales y mejora la mantenibilidad.

3.2. Componentes parcialmente acoplados

Componente	Observación
Orchestrator	Coordina múltiples servicios
Modelo de respuesta	Dependencia del flujo de negocio actual

Aunque existe cierto nivel de acoplamiento lógico en la orquestación, esto es natural en arquitecturas de coordinación centralizada.

4. Escalabilidad y Soporte para Nuevos Clientes

La arquitectura soporta crecimiento mediante:

4.1. Incorporación de nuevas compañías

Se logra agregando nuevas configuraciones en:

- categories.json
- delegaciones.json

Sin modificar código fuente.

4.2. Nuevas categorías

Las categorías son dinámicas y configurables por empresa. Esto permite:

- Diferentes dominios de negocio
- Personalización por cliente
- Evolución independiente

4.3. Reglas de prioridad variables

La solución permite evolucionar hacia:

- Reglas configurables
- Motores de reglas
- Integraciones externas
- Modelos híbridos

4.4. Integración con plataformas externas

El diseño desacoplado facilita integración futura con:

- CRM
- ERP
- Plataformas BPO
- APIs externas

4.5. Escalabilidad de volumen

FastAPI permite procesamiento eficiente y asíncrono.

En escenarios de alto volumen, la solución puede evolucionar utilizando:

- Colas de mensajería
- Workers distribuidos
- Balanceadores de carga
- Caché distribuido

5. Estrategia de Inteligencia Artificial

La solución implementa un enfoque híbrido.

5.1. Componentes IA

Se utiliza un modelo LLM para:

- Clasificación
- Priorización
- Generación de justificaciones

Proveedor utilizado:

- Groq

Modelo:

- Llama 3.3 70B Versatile

5.2. Justificación de la estrategia

El uso de LLM permite:

- Procesar lenguaje natural
- Manejar texto no estructurado
- Reducir reglas manuales
- Adaptarse a múltiples dominios

La combinación con configuraciones externas proporciona:

- Flexibilidad
- Control empresarial
- Adaptabilidad por cliente

6. Variables de Entorno

La solución utiliza variables de entorno para proteger credenciales.

Archivo `.env`:

```
GROQ_API_KEY=xxxxxxx
```

Esto evita exponer información sensible en el código fuente.

7. Ejecución Local

La solución puede ejecutarse localmente mediante:

```
uvicorn app.main:app --reload
```

Swagger:

- <http://127.0.0.1:8000/docs>

8. Propuesta de Despliegue Cloud

Aunque el despliegue cloud no es requerido, la arquitectura permite evolucionar fácilmente a un ambiente productivo.

8.1. Propuesta cloud

- Docker
- Kubernetes
- Nginx o API Gateway Cloud
- Load Balancer
- RabbitMQ o Kafka
- Celery
- Redis
- PostgreSQL
- Prometheus + Grafana
- AWS Secrets Manager / Azure Key Vault

9. Conclusión

La solución propuesta fue diseñada priorizando modularidad, escalabilidad y mantenibilidad.

El uso de microservicios ligeros, configuración externa y componentes desacoplados permite incorporar nuevas empresas, categorías y reglas de negocio con bajo impacto técnico.

La arquitectura también permite evolucionar fácilmente hacia escenarios cloud y de alto volumen, manteniendo flexibilidad para integraciones futuras y crecimiento empresarial.